FUNDACIÓN UNIVERSITARIA TECNOLÓGICO COMFENALCO
FACULTAD DE INGENIERÍA • PROGRAMA DE INGENIERÍA DE SISTEMAS
VII SEMESTRE — PROYECTO DE AULA (SEGUNDO SEGUIMIENTO)

CLINICAAPP: PLATAFORMA INTEGRAL PARA LA GESTIÓN VETERINARIA

ENTREGABLE OFICIAL: DESARROLLO WEB AVANZADO
Docente Evaluador: Ing. Fabio García Ramírez

Autores / Investigadores: Luis Cárdenas, Cristóbal Villamil, Jose Castillo, Daniel Gutiérrez.
Fecha: Octubre de 2026 • Cartagena de Indias D.T. y C.
Repositorio Oficial: https://github.com/cardenaswalker2/ClinicaApp.git

1. Introducción y Alcance en Desarrollo Web Avanzado

El presente documento consolida la entrega formal y auditable para la asignatura de Desarrollo Web Avanzado en el marco del proyecto de aula ClinicaApp. Se estructura una arquitectura modular y desacoplada en 5 capas fundamentada en Spring Boot 3.5 con Java 21 LTS, potenciada con el asistente conversacional inteligente Nova Brain integrado mediante APIs RESTful seguras con Modelos de Lenguaje Masivos (LLM Llama 3.3 70B vía Groq Cloud), respaldo por motor heurístico local (MOTOR_LOGICO), control de acceso estricto (RBAC de 6 roles), autenticación federada OAuth2/OIDC y reconocimiento biométrico facial 128D.

2. Diagrama de Arquitectura de la Aplicación (Spring Boot 3.5 + LLM)

En concordancia con los lineamientos de la guía y tomando como referencia la estructura del proyecto Educhat-notes, la arquitectura de ClinicaApp desacopla las responsabilidades en 5 capas cardinales:

Capa	Componentes Principales en ClinicaApp	Responsabilidad Técnica
Capa 1: Presentación / Frontend	Thymeleaf UI, JavaScript reactivo (app.js, chat.js), Web Speech API (Voz STT/TTS), Bootstrap 5.3 adaptable.	Renderizado de vistas del cliente, captura de audio del usuario, sintetización por voz y llamadas fetch asíncronas REST.
Capa 2: Controller (@RestController / @Controller)	NovaAIController (/api/nova-brain/think), UsuarioController, ClinicaController, CitaController, OptimizacionController.	Recepción de peticiones HTTP, validación de DTOs (NovaRequest), orquestación y respuesta JSON (NovaResponse).
Capa 3: Service (@Service - Lógica de Negocio)	NovaAIService, UsuarioServiceImpl, CitaService, OptimizacionServiceImpl, MaintenanceFilter, NetworkDeviceFilter.	Inyección de dependencias, ensamblado de prompt contextual RAG, gestión de fallback MOTOR_LOGICO y biometría facial.

Capa 4: Repository (Spring Data MongoDB)	UsuarioRepository, MascotaRepository, ClinicaRepository, CitaRepository, ServicioRepository, LogInferenciaRepository.	Persistencia no relacional documental en MongoDB Atlas 7.0, consultas por Aggregation Framework e índices optimizados.
Capa 5: Proveedor LLM & Analítica Externa	Groq Cloud API (Llama 3.3 70B Versatile vía ${GROQ_API_KEY}), Datasets CSV Kimball en docs/powerbi_datawarehouse/.	Inferencia generativa ultra-rápida en la nube y repositorio de 3.469 hechos preparados para cuadros de mando.

3. Prototipo de Clases del Agente Inteligente (Nova Brain)

Siguiendo el patrón arquitectónico del agente conversacional, se estructuran las siguientes clases:

• NovaAIController.java: Controlador REST que expone el endpoint /api/nova-brain/think. Extrae la identidad del usuario desde SecurityContextHolder, consulta en MongoDB sus mascotas registradas y citas pendientes, y construye el prompt del sistema.

• NovaRequest.java / NovaResponse.java: Objetos de transferencia de datos (DTOs) para tipar fuertemente las solicitudes, intenciones detectadas (TRIAGE_EMERGENCIA, INFO_SERVICIOS, AGENDAMIENTO) y acciones recomendadas.

• LogInferencia.java / LogInferenciaRepository.java: Entidad y repositorio para auditoría y observabilidad en MongoDB, guardando: usuarioId, prompt, respuesta, tokens consumidos y latencia en milisegundos.

• Motor de Contingencia (MOTOR_LOGICO): En caso de corte de conexión externa o falta de API key en producción, un analizador regex heurístico local responde inmediatamente garantizando disponibilidad 24/7.

4. Vistas Frontend e Integración con ChatBot / Asistente de Voz

• Vista asistente_voz.html: Widget moderno con Glassmorphism integrado en la interfaz de ClinicaApp.

• Reconocimiento de Voz (STT): Utilización de Web Speech API (webkitSpeechRecognition) permitiendo consultas habladas del usuario en lenguaje natural.

• Sintetizador de Voz (TTS): Respuesta hablada del agente inteligente mediante SpeechSynthesisUtterance.

• Validaciones Reactivas: Manejo visual de estados de carga, chips de preguntas rápidas y validación estricta contra envíos vacíos.

5. Seguridad Perimetral, Spring Security, RBAC y Biometría Facial

• SecurityConfig.java: Definición del SecurityFilterChain, protección contra ataques CSRF, y políticas de sesión.

• Control de Acceso Basado en Roles (RBAC): 6 roles estrictos: ROLE_ADMIN, ROLE_CLINICA, ROLE_VETERINARIO, ROLE_RECEPCIONISTA, ROLE_AUXILIAR y ROLE_CLIENTE.

• Autenticación Federada: Integración OAuth2 / OpenID Connect (OIDC) con Google y GitHub.

• Biometría Facial 128D: Algoritmo implementado en UsuarioServiceImpl.java que calcula la distancia euclidiana entre el vector facial registrado del usuario y la captura de cámara en vivo, validando el acceso con umbral estricto (< 0.38).

6. Modelo de Datos y Diccionario E-R (MongoDB Atlas 7.0)

Colecciones operacionales verificadas: usuarios (identidad y biometría), mascotas (perfil biológico), clinicas (georreferenciación y estado), citas (reservas médicas), servicios (catálogo de procedimientos), visitas (diagnóstico y facturación) y logs_inferencia (trazabilidad de IA).